

LAPORAN PRAKTIKUM 2
STATISKA DAN PROBABILITAS



Disusun Oleh :

Muhammad Harits Arrozzaq (2400016087)

Kelas Praktikum/Semester :

C/III

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS SAINS & TEKNOLOGI TERAPAN
UNIVERSITAS AHMAD DAHLAN
YOGYAKARTA
2025

A. Dasar Teori

Transformasi data adalah proses mengubah skala atau bentuk distribusi data menjadi distribusi yang lebih sesuai untuk analisis statistik. Hal ini dilakukan untuk memenuhi asumsi dari metode statistik tertentu atau untuk membuat data lebih sesuai dengan model yang digunakan. Transformasi data biasanya dilakukan ketika data tidak memenuhi asumsi dasar analisis statistik tertentu, seperti asumsi normalitas, homoskedastisitas, atau linearitas. Dengan transformasi yang tepat, kita dapat membuat data lebih sesuai dengan asumsi tersebut dan meningkatkan interpretabilitas hasil analisis.

B. Langkah – Langkah Percobaan

1. Logaritmik

```
[2]: import numpy as np

data_pengeluaran = [100, 200, 150, 300, 250, 400, 350, 500]

data_log = np.log10(data_pengeluaran)

print("Data Asli:", data_pengeluaran)
print("Data setelah Transformasi Logaritmik:", data_log)

Data Asli: [100, 200, 150, 300, 250, 400, 350, 500]
Data setelah Transformasi Logaritmik: [2.          2.30103    2.17609126 2.47712125 2.39794001 2.60205999
 2.54406804 2.69897   ]
```

2. Akar Kuadrat

```
[23]: import numpy as np

data_tinggi_badan = [150, 155, 160, 165, 170, 175, 180, 185, 190, 195]

data_tinggi_badan_akar = [np.sqrt(x) for x in data_tinggi_badan]

print("Data Tinggi Badan Asli:", data_tinggi_badan)
print("Data Tinggi Badan Setelah Transformasi Akar Kuadrat:", data_tinggi_badan_akar)

Data Tinggi Badan Asli: [150, 155, 160, 165, 170, 175, 180, 185, 190, 195]
Data Tinggi Badan Setelah Transformasi Akar Kuadrat: [np.float64(12.24744871391589), np.float64(12.449899597988733), np.float64(12.6491106406
73518), np.float64(12.84523257866513), np.float64(13.038404810405298), np.float64(13.228756555322953), np.float64(13.416407864998739), np.flo
at64(13.601470508735444), np.float64(13.784048752090222), np.float64(13.96424004376894)]
```

3. Reciprocal

```
[21]: def reciprocal_transform(data):
    reciprocal_data = [1 / x for x in data]
    return reciprocal_data

data_awal = [2, 4, 6, 8, 10]

data_reciprocal = reciprocal_transform(data_awal)

print("Data Awal:", data_awal)
print("Data setelah Transformasi Reciprocal:", data_reciprocal)

Data Awal: [2, 4, 6, 8, 10]
Data setelah Transformasi Reciprocal: [0.5, 0.25, 0.16666666666666666, 0.125, 0.1]
```

4. Box-Cox

```
[24]: from scipy.stats import boxcox
import numpy as np

data_pengeluaran = [100, 200, 150, 300, 250, 400, 350, 500]

data_transformed, lam = boxcox (data_pengeluaran)

print ("Data Pengeluaran Asli:", data_pengeluaran)
print("Data Pengeluaran setelah Transformasi Box-Cox:", data_transformed)
print ("Parameter Lambda untuk Transformasi Box-Cox:", lam)

Data Pengeluaran Asli: [100, 200, 150, 300, 250, 400, 350, 500]
Data Pengeluaran setelah Transformasi Box-Cox: [18.88139466 27.81983945 23.7195968 34.73497319 31.44627977 40.59535799
37.76700778 45.77854774]
Parameter Lambda untuk Transformasi Box-Cox: 0.5152861056316803
```

C. Langkah – Langkah Praktikum

Soal 1 :

1. Lakukan transformasi data menggunakan logaritma natural (log base e) dan square root pada dataset berat badan tersebut.

```
[13]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Dataset Berat Badan (dalam kg)
data_berat_badan = [
    42, 56, 70, 68, 50, 65, 80, 45, 75, 62, 48, 52, 90, 44, 78, 66, 53, 72, 61, 54,
    57, 85, 67, 69, 51, 58, 55, 60, 73, 64, 59, 49, 88, 92, 74, 84, 47, 83, 71, 63,
    77, 79, 82, 81, 86, 87, 89, 91, 93, 94, 95, 40, 43, 76, 46, 99, 100, 110, 105, 102
]

# 1. Transformasi Data
# Transformasi Logaritma Natural (ln atau log base e)
df['Log_Berat_Badan'] = np.log(df['Berat_Badan_Asl'])

# Transformasi Akar Kuadrat (Square Root)
df['Sqrt_Berat_Badan'] = np.sqrt(df['Berat_Badan_Asl'])

print("DataFrame dengan Data Transformasi (5 baris pertama):")
print(df.head())
print("-" * 50)
```

DataFrame dengan Data Transformasi (5 baris pertama):

	Berat_Badan_Asl	Log_Berat_Badan	Sqrt_Berat_Badan
0	42	3.737670	6.480741
1	56	4.025352	7.483315
2	70	4.248495	8.366600
3	68	4.219508	8.246211
4	50	3.912023	7.071068

2. Bandingkan distribusi data asli dengan data yang telah ditransformasikan melalui histogram dan boxplot.

```
# 2. Visualisasi Perbandingan Distribusi
# Histogram
plt.figure(figsize=(10, 5))

# Histogram Data Asli
plt.subplot(1, 3, 1)
plt.hist(df['Berat_Badan_Asl'], bins=10, edgecolor='black', color='skyblue')
plt.title('Histogram: Berat Badan Asli')
plt.xlabel('Berat Badan (kg)')
plt.ylabel('Frekuensi')

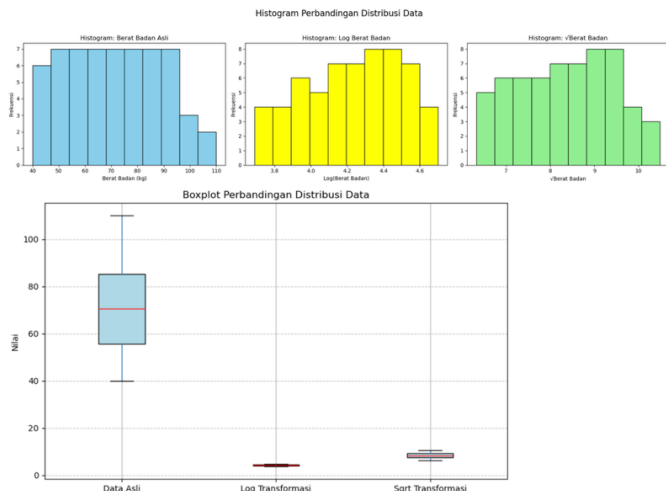
# Histogram Log Berat Badan
plt.subplot(1, 3, 2)
plt.hist(df['Log_Berat_Badan'], bins=10, edgecolor='black', color='yellow')
plt.title('Histogram: Log Berat Badan')
plt.xlabel('Log(Berat Badan)')
plt.ylabel('Frekuensi')

# Histogram Sqrt Berat Badan
plt.subplot(1, 3, 3)
plt.hist(df['Sqrt_Berat_Badan'], bins=10, edgecolor='black', color='lightgreen')
plt.title('Histogram: vBerat Badan')
plt.xlabel('vBerat Badan')
plt.ylabel('Frekuensi')

plt.suptitle('Histogram Perbandingan Distribusi Data', fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

# Boxplot
plt.figure(figsize=(10, 6))

df_plot = df[['Berat_Badan_Asl', 'Log_Berat_Badan', 'Sqrt_Berat_Badan']]
df_plot.boxplot(
    vert=True,
    patch_artist=True,
    boxprops=dict(facecolor='lightblue', color='black'),
    medianprops=dict(color='red')
)
plt.title('Boxplot Perbandingan Distribusi Data')
plt.ylabel('Nilai')
plt.xticks(
    ticks=[1, 2, 3],
    labels=['Data Asli', 'Log Transformasi', 'Sqrt Transformasi']
)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



3. Jelaskan bagaimana transformasi data tersebut mengubah distribusi data awal.

- Skewness data asli adalah positif, menandakan ekor ke kanan (Data Asli: ~ 0.35).
- Skewness Log Berat Badan mendekati nol, menandakan distribusi menjadi sangat simetris (Log: ~ -0.08).
- Skewness Sqrt Berat Badan berkurang dari data asli tetapi lebih tinggi dari Log, (Sqrt: ~ 0.16) menandakan masih ada sedikit kemiringan positif tersisa.

Soal 2 :

```
[27]: import pandas as pd
import numpy as np

# Dataset Nilai Ujian (0-100) dari 50 siswa
nilai_ujian = [
    58, 67, 45, 89, 72, 60, 76, 93, 55, 48, 62, 85, 79, 56, 41, 90, 77, 54, 68, 82,
    46, 73, 57, 92, 81, 53, 66, 74, 64, 52, 91, 78, 49, 87, 88, 50, 69, 84, 43, 65,
    83, 70, 44, 61, 75, 80, 71, 63, 47, 51
]

# Membuat DataFrame
df = pd.DataFrame({'Nilai_Asl': nilai_ujian})

# Analisis Data Asli
mean_asli = df['Nilai_Asl'].mean()
std_dev_asli = df['Nilai_Asl'].std(ddof=1)

print("--- Data Asli ---")
print(f"Mean (Rata-rata): {mean_asli:.2f}")
print(f"Standard Deviation (Simp. Baku): {std_dev_asli:.2f}")

# Transformasi Z-Score
df['Z_Score'] = (df['Nilai_Asl'] - mean_asli) / std_dev_asli

mean_z = df['Z_Score'].mean()
std_dev_z = df['Z_Score'].std(ddof=1)

print("--- Data Setelah Transformasi Z-Score ---")
print(df.head(10))
print("-" * 40)
print(f"Mean (Z-Score): {mean_z:.2f}")
print(f"Standar Deviasi (Z-Score): {std_dev_z:.2f}")

--- Data Asli ---
Mean (Rata-rata): 67.28
Standard Deviation (Simp. Baku): 15.22
--- Data Setelah Transformasi Z-Score ---
   Nilai_Asl  Z_Score
0         58 -0.609796
1         67 -0.018399
2         45 -1.464037
3         89  1.427239
4         72  0.310155
5         60 -0.478375
6         76  0.572998
7         93  1.690082
8         55 -0.806929
9         48 -1.266904

-----
Mean (Z-Score): -0.00
Standar Deviasi (Z-Score): 1.00
```

Penjelasan :

Setelah dilakukan transformasi Z-score, nilai mean dataset berubah dari 67.28 menjadi 0, dan standar deviasi dari 15.22 menjadi 1. Perubahan ini menunjukkan

bahwa data telah distandarkan, sehingga setiap nilai kini merepresentasikan jarak dalam satuan simpangan baku dari rata-rata. Transformasi ini tidak mengubah bentuk distribusi data, tetapi hanya mengubah skala agar dapat dibandingkan secara proporsional antar variabel.

Soal 3 :

```
import pandas as pd
import numpy as np
from scipy.stats import shapiro
import matplotlib.pyplot as plt

# Dataset Penjualan Produk Bulanan (dalam ribuan unit)
penjualan_data = [
    1020, 980, 1200, 1120, 1340, 890, 1010, 950, 1050, 1230, 1110, 990, 1130, 970, 1320,
    900, 1090, 1000, 1070, 1080, 1150, 1100, 1250, 980, 1020, 930, 1200, 1030, 1270,
    1060, 1170, 1010, 1190, 1260, 990, 1300, 940, 1040, 1280, 1090, 1160, 1080, 950,
    1290, 1240, 950, 1050, 1220, 970, 1100, 1180, 990, 1020, 1110, 900, 1210, 950, 1060,
    1130, 1220
]

# Membuat DataFrame
df = pd.DataFrame({'Penjualan_Aslis': penjualan_data})

# Transformasi Data
df['Log_Penjualan'] = np.log(df['Penjualan_Aslis'])

print("5 baris pertama data setelah transformasi:")
print(df.head())
print("-" * 60)

stat_asli, p_asli = shapiro(df['Penjualan_Aslis'])
stat_log, p_log = shapiro(df['Log_Penjualan'])

print("Hasil Uji Normalitas Shapiro-Wilk (H0: Data berdistribusi Normal):")
print(f"Data Asli: Statistik W = {stat_asli:.4f}, p-value = {p_asli:.4f}")
print(f"Data Log-Transformasi: Statistik W = {stat_log:.4f}, p-value = {p_log:.4f}")
print("-" * 60)

print("Analisis Skewness (Kemiringan):")
print(f"Skewness Data Asli: {df['Penjualan_Aslis'].skew():.4f}")
print(f"Skewness Data Log-Transformasi: {df['Log_Penjualan'].skew():.4f}")
print("-" * 60)

plt.figure(figsize=(12, 5))

# Histogram Data Asli
plt.subplot(1, 2, 1)
plt.hist(df['Penjualan_Aslis'], bins=10, edgecolor='black', color='skyblue')
plt.title('Histogram Penjualan Asli')
plt.xlabel('Penjualan (ribuan unit)')
plt.ylabel('Frekuensi')

# Histogram Data Log-Transformasi
plt.subplot(1, 2, 2)
plt.hist(df['Log_Penjualan'], bins=10, edgecolor='black', color='red')
plt.title('Histogram Log Penjualan')
plt.xlabel('Log(Penjualan)')
plt.ylabel('Frekuensi')

plt.suptitle('Perbandingan Distribusi Penjualan Sebelum dan Sesudah Log-Transformasi', fontsize=14)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

5 baris pertama data setelah transformasi:

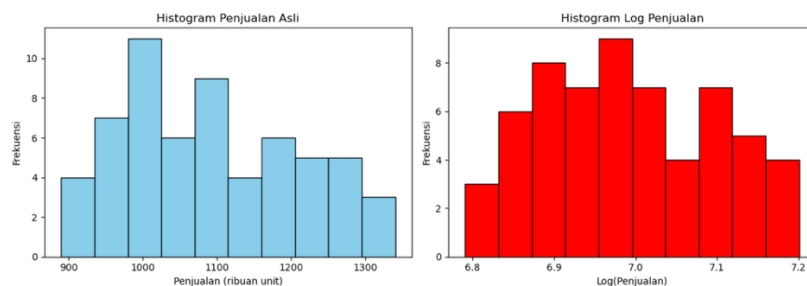
	Penjualan_Aslis	Log_Penjualan
0	1020	6.927558
1	980	6.887553
2	1200	7.090077
3	1120	7.021084
4	1340	7.200425

Hasil Uji Normalitas Shapiro-Wilk (H0: Data berdistribusi Normal):
Data Asli: Statistik W = 0.9642, p-value = 0.0752
Data Log-Transformasi: Statistik W = 0.9702, p-value = 0.1485

Analisis Skewness (Kemiringan):

Skewness Data Asli: 0.2893
Skewness Data Log-Transformasi: 0.1280

Perbandingan Distribusi Penjualan Sebelum dan Sesudah Log-Transformasi



Penjelasan :

Setelah dilakukan transformasi logaritma, distribusi data penjualan menjadi lebih mendekati normal. Nilai skewness menurun dari 0.2893 menjadi 0.1280 dan p-value uji Shapiro-Wilk meningkat dari 0.0752 menjadi 0.1485, menunjukkan data hasil transformasi lebih simetris dan memenuhi asumsi normalitas. Secara visual, histogram log-penjualan tampak lebih seimbang dibanding data asli, sehingga transformasi log efektif dalam mengurangi kemiringan dan menstabilkan distribusi data.

D. Kesimpulan

transformasi data berfungsi untuk memperbaiki bentuk distribusi agar lebih sesuai dengan asumsi normalitas dan memudahkan analisis statistik. Transformasi logaritma dan akar kuadrat mampu mengurangi kemiringan data (skewness) serta membuat distribusi lebih simetris, sedangkan transformasi Z-score menstandarkan data dengan mean menjadi 0 dan standar deviasi menjadi 1 tanpa mengubah bentuk distribusi. Secara keseluruhan, penerapan transformasi data membuat data lebih stabil, seimbang, dan siap digunakan untuk analisis statistik lanjutan.